Handoff: Babbage's Hall

For a new chat: how to work, build, test, and what's next · 14 November 2026

Read this first in a new chat. It says how William likes to work, where everything is, how to build and test, what has been learned, and what to do next. The list of machines is `MACHINES.md`; the hall itself is described in `README.md`. This file is updated whenever a chat ends.

1. What this is

- **Babbage's Hall** (`demo/WyomingWill/BabbagesHall/`), a Victorian engineering hall in the Bullet Physics Playground (bpp, **github.com/bullet-physics-playground/bpp**; William pushes to the branch **`test-harness`** (since 2026-11-11), which Jakob merges into `master`): cast iron, brass, a glass roof, holding **machines that compute**. A separate museum from William's Euler's Gallery (`../EulersGallery/`), built on its machinery (separate worlds, sandboxes, walking with Tab).
- **Every machine can be 3D printed and built.** Each exhibit's parts are OpenSCAD sources (the single source for the simulation and the prints), a `make-stls.sh` makes the STL files, and its README lists what to buy and how to assemble it. What is printed is what is simulated: no shortcuts in the parts that compute, and anything idealised is said on the placard and in FINDINGS.
- **The walk asks more of you as you go:** from what asks least (one idea, little mechanism) to what asks most, Babbage's Difference Engine at the far end. `MACHINES.md` has the order and why (re-ordered 2026-11-14: it was by interest alone).
- Built so far, in walking order: **the Domino Half Adder** (west 1), **the Differential** (east 1), **the Pascaline** (west 2), **the Galton board** (west 3, since 2026-11-12), **the splitter board** (east 3, facing it, since 2026-11-14), **Lehmer's number sieve** (west 4), and the **Digi-Comp II** (east 4: it counts, adds, multiplies, subtracts and divides, runs programs, and in AUTO runs a multiply by itself, or a division to OVERFLOW's halt; its weight and fly are parts: a cup of balls and a bought rotary damper; each program has only its parts on the board, and the operator's hand is shown). Covered bays wait for Amsler's planimeter (east 2), Kelvin's tide predictor (west 5), the Differential Analyser (east 5) and the Difference Engine (west 6): eleven bays, six pairs.
- William M. Spears (`williammspears@gmail.com`) owns the WyomingWill demos; Jakob (koppi) wrote bpp. Clocks are William's own separate collection.

2. How William likes to work

- **Deliver every change as a patch plus the git commands.** He applies patches in a checkout called `bpp-push`, pushes, then pulls in his installation checkout (which has `pull.ff only`; never apply patches there). Give the commands as below, and say if bpp must be rebuilt (only when C++ changed).

```
git pull
git am ~/Downloads/<name>.patch
git push
```

- **Every patch includes all its documents, PDFs too.** Every `.md` has a typeset `.pdf` beside it (`./make-pdfs.sh`, or `tools/md2pdf.py` for one); rebuild the PDF whenever its `.md` changes. Euler's

Gallery and the Rec Room have README.pdfs made with `../ArtificialPhysics/tools/md2pdf.py`.

- **Build patches on `origin/test-harness`**, where he pushes (a clone fetches only `master`: `git fetch origin test-harness:refs/remotes/origin/test-harness`). To check something landed, look there, not on `master`.
- **Before sending a patch:** check it applies to the current `origin/test-harness` (fetch first; a patch built on one he hasn't applied fails, as the first `MACHINES.md` patch did); no trailing spaces (`git diff --check`); `tools/check-all.sh` passes; the visuals looked at (`tools/shots.sh`). A re-sent patch gets a new file name, so a stale download can't be applied.
- Commits are authored `William M. Spears <williammspears@gmail.com>`, with the Co-Authored-By trailer for Claude. Stage only the files you changed.
- He looks at the exhibits in the GUI and reports what he sees with screenshots: take his reports seriously and reproduce them before answering, and measure when the pictures don't settle it.
- Plain explanations; measure before claiming; say what was and wasn't tested.
- **His bpp is built with Lua 5.2;** a cloud workspace's is usually 5.1. Write Lua that runs on both: no bare `setfenv`/`getfenv` (give `loadfile` its environment with `"t", _ENV` on 5.2, as the exhibits' table loaders do), no `goto`. `tools/check-lua52.sh` checks the one place this has bitten (the table loader) under a standalone `lua5.2`.

3. Where everything is

Path	What
`hall.lua`	The hall: architecture, the EXHIBITS list (walking order, placards, `floorY`), sandboxes, walking and camera flights, resting, the cost meter
`exhibit-table.lua`	The standard exhibit table (README "Exhibit standard")
`MACHINES.md`	All 48 machines considered, in walking order (five tiers), scored for interest and complexity, with how each would go in bpp and be printed; the rope test
`README.md`	The hall: bays, keys, how it works, the cost meter, the exhibit standard, notes for exhibit authors
`HANDOFF.md`	This file
`make-pdfs.sh`, `tools/md2pdf.py`	The PDFs
`make-labels.py`, `make-lecterns.py`, `hall-meshes/`	The placards' lettering (Pillow, NumPy, DejaVu fonts); the lecterns' cards (PNG)
`tools/check-all.sh`	Every exhibit's tour, on its own and in the hall
`tools/shots.sh`, `tools/keys.sh`	Screenshots on a virtual screen; with keys, hover, camera flights
`tools/check-lua52.sh`	The Lua 5.2 sandbox check
`tools/rope-test.lua`	The rope experiment (for the tide predictor)
`exhibits/DigiComp2/tests/toggle.lua`, `tests/switch.lua`, `tests/junction.lua`	One Digi-Comp flip-flop alone (with its foot, done hole and clear lane); one switch; the piece of board where the memory meets the accumulator
`exhibits/DigiComp2/MANUAL.md`, `manual/`	The Digi-Comp's illustrated manual; `manual/make-figures.sh` redraws its figures from the geometry (lua, python3, matplotlib). `tools/md2pdf.py` puts a `.md` file's images into its PDF
`exhibits/<Name>/`	One folder per exhibit: its script, `parts/` (OpenSCAD), `make-stls.sh`, `README.md`/`.pdf`, `FINDINGS.md`/`.pdf`

Related: `../EulersGallery/` (the gallery the hall's machinery came from; its tables still follow its own sizes, see §7), `../RecRoom/` (games room: the cost meter and suspending came from there), `demo/claude/digicomp.lua` (an incomplete Digi-Comp II, a starting point), `module/scad/gearsv50.lua` and the Differential's `parts/gearsv50.scad` (bevel gears).

4. Building, running, testing

```
git clone https://github.com/bullet-physics-playground/bpp ~/bpp && cd ~/bpp
apt-get update
apt-get install -y libbullet-dev libluabind-dev liblua5.1-0-dev liblua5.2-dev \
  libqglviewer-dev-qt5 libsdl2-mixer-dev libassimp-dev qtbase5-dev libglew-dev \
  xvfb xdotool imagemagick openscad lua5.2 povray
pip install --break-system-packages trimesh manifold3d markdown playwright
mkdir -p build && cd build && qmake ../bpp.pro CONFIG+=release && make -j2 release
```

That gives `build/release/bpp` (a few minutes to ten). The tools find it there, in `release/`, or on the path; or set `BPP`.

- **Headless:** from the top folder, `build/release/bpp -f demo/WyomingWill/BabbagesHall/hall.lua -n 600`. `HALL_START=pascaline` starts at an exhibit. The exhibits run on their own the same way, each with a tour that prints `RESULT <name> tour: PASS`.
- **GUI on a virtual screen:** `tools/shots.sh` (see its header); `LIBGL_ALWAYS_SOFTWARE=1`, so drawing is slow and the simulation runs slower than real time there. In the GUI, `print` goes to bpp's console pane, not the terminal: write to a file (the meter's `HALL_METER_FILE`, the flights' `HALL_FLY_LOG=file`) to read it.
- **Printed parts:** each exhibit's `make-stls.sh` (OpenSCAD 2021.01 here; William's is newer: avoid `assign()` and `polyhedron(triangles=)`). Check every STL is one watertight solid (trimesh), and that parts don't overlap at rest (manifold3d); OpenSCAD renders pictures under `xvfb-run`.
- **POV-Ray:** `build/release/bpp -f ... -n 2 -e` writes `export/<script>/` (untracked; delete it after); render with `povray +I<script>.pov +K2 +W800 +H450 +L<bpp folder>`.

5. What has been learned (bpp and Bullet)

- **OpenSCAD parts** are cached by the text asked for, not by the files' contents: put a checksum of every file the part includes into the text (each exhibit's `checksum()`). Set `collides` before `v:add`, the transform after. A body's origin must be its centre of mass.
- **`polyhedron()` faces go clockwise seen from outside.** OpenSCAD 2021's CGAL forgave the other way when it joined pieces; William's newer OpenSCAD (Manifold) doesn't, and such a part comes out inside out: drawn with dark slivers when bpp's shadows are off (the shader hides it), and a bad STL. Check a new polyhedron's signed volume (trimesh, or manifold3d on its points). Fixed 2026-11-04 in the Differential's gears and the sieve's cam ring.
- **Two bpp processes building the same new part at once** can collide in the mesh cache: run tests one at a time after a part changes.
- **Gears:** mesh-on-mesh (GImpact) bevel teeth jam; a compound of convex hulls (one per tooth plus the body, from the same SCAD source) meshes well (`exhibits/Differential/FINDINGS.md`).
- **Hinge angles** (`getHingeAngle`) are clockwise-positive as seen down the hinge's axis, the opposite of a right-handed turn. A hinge's motor speed counts the same way as its angle (measured on the Digi-Comp's wheel).

- **A pin's friction** can be had from a hinge's motor held at no speed with a small impulse a step (`enableAngularMotor(true, 0, torque * step)`); the Digi-Comp's rockers use it, from PLA on PLA at the boss.
- **Over-centre parts** (a rocker whose centre of mass is above its pin) rest in a very shallow dip: with no friction they bounced back out of it, with pin friction they stop wherever they're left. Give them weight high up (the Digi-Comp's rockers have a steel ball pressed in) and test both extremes.
- **Walls as boxes** run past their ends by half their thickness if made `length + thickness` long, so the joints are solid: check where such an end meets a moving part (the Digi-Comp's feeder wheel rubbed two), or narrows a channel (two jammed the Digi-Comp's clear lane). Mitre a lane's walls at its bends (`laneWalls` in the Digi-Comp's geometry).
- **A ball pushing a part out of its way** pushes it only until it can squeeze past, which may be nowhere near where the part should end up: to push it all the way, guide the ball (the Digi-Comp's clear lane narrows beside each foot).
- **Two ways into one place:** trace the ball that comes in first (its position every frame) before giving the other one a way in; a ball rolling down a wall must pass under the ends of the walls above it with room for the ball (the Digi-Comp's joining funnels, `FINDINGS.md`). Lanes on one level can't cross: their order is fixed by where they start and end, so plan a board in a drawing of every wall first.
- **A part with holes** (the Digi-Comp's upper plate) is a static body of boxes round the holes, drawn not by them (`transparency = 1`) but by the OpenSCAD part. `transparency` near 0.35 draws a see-through plate well.
- **Luabind objects** from `v:eachContact` can't be compared with `==`; look them up as table keys (`names[obj]`).
- **Small steps:** the exhibits take 2400 steps a simulated second, stepped by hand in `preSim` (`common.setTiming(STEP, 0, STEP)`); the teeth and the dominoes need it. Sleeping is off for moving parts (state 4), so each exhibit **rests** (stops stepping) once nothing moves; it changes nothing computed.
- **Springs** that matter are simulated as the printed part: the Pascaline's detent is a body on a hinge turned back by the leaf's stiffness from PLA (3EI/L), and its working range of thickness was found by sweeping it.
- **A part driven by a weight through a damper** moves slowly wherever only a small force drives it: the Digi-Comp's escapement, whose pin must creep off the anchor's second finger, took up to 1.8 s to let go with the damper where the torque-and-cap had taken half a second. Measure such waits with the real drag, at both ends of its tolerance.
- **An object's `pos` (and `trans`) is its drawing's**, which follows its body only after a step: read a body that has just been placed from the body (`getCenterOfMassPosition()`).
- **`getCenterOfMassTransform()` is the body's own transform**, not a copy (and `getCenterOfMassPosition()` its own vector): keep a pose by copying it (`btTransform(tr:getRotation(), btVector3(p.x, p.y, p.z))`). Kept as is, it moves with the body (the Digi-Comp's parts were "put back" where they had been sent). An object's `.trans` is a copy.
- **Taking parts away finds hidden jobs.** A wall drawn as one part's may do another's work (the Digi-Comp's clear lane steered done balls; a done lane's wall was its hole's backstop). Test each reduced machine, in AUTO too: in MANUAL a ball stopped in the wrong place often still counts as done.
- **Impulses applied in `preSim` before a step** (a cord's pull, a drag) change a held body's velocity for an instant, until the step's contacts take it back: read speeds for "is it moving?" before applying them (the Digi-Comp's hand thought the held wheel was turning, and never began).
- **Rope:** none built in; a rope of beads on ball joints works if the rope isn't far lighter than what hangs on it (`MACHINES.md`, "Rope").

- **Mouse:** scripts get `onHover` (after half a second still, no button down) but no clicks; keys override bpp's own one-letter keys.
- `btTransform` has no `getOrigin` in bpp's Lua; `.trans`, `:mult` and `getCenterOfMassTransform` are what there is.
- **Compounds:** a `btCompoundShape` in bpp's Lua takes only a few dozen children (each costs one of luabind's few dependency slots; past them bpp stops with an assertion in `object_rep.cpp`). Build big bodies as trees of small compounds, neighbours together (Lehmer's sieve's `tree()`), and don't add and remove children over and over: move them (`updateChildTransform`), as the sieve's pegs do.
- **Cylinders:** bpp's Lua has `btCylinderShapeX` and `btCylinderShapeZ` but no constructor for the upright `btCylinderShape`: stand a Z one up.
- **Convex pieces** from a 2D outline: `exhibits/LehmerSieve/parts/make-geometry.py` (shapely's constrained triangulation, merged while convex); check that the pieces' areas add up to the outline's. Corners inside an outline cost pieces; draw holes and inner arcs coarser for the contacts than for the print.
- **An OpenSCAD part made a body** must be set to collide before `v:add` (the exhibits' `part()` turns collisions off).
- `v:eachContact(fn)` calls `fn(a, b, x, y, z, nx, ny, nz, distance, impulse)` for every contact: the quickest way to find what jams.
- **Geneva drives** index well but their locking disc only bounds a star's turn (about a degree either way): a sprung detent per star puts it on its place, and the disc's relief must clear the star's lead and lag (`exhibits/LehmerSieve/FINDINGS.md`).
- `.gitignore` at the top ignores `*.scad`: the hall's own `.gitignore` has `!*.scad`, and STL files are not kept (each `make-stls.sh` makes them).
- **Balls in two dimensions jam** (the Galton board, `FINDINGS.md`): a hopper of balls between two plates, emptying through a neck, clogged at three balls wide and at five; a ball perched on a pin under the neck held up an arch or a zigzag column; taps (rocking the board) did not clear it. A neck wide enough flooded the pins, and the crowded balls spread far too wide. A slotted wheel metering one ball at a time is what worked.
- **A mechanism hands every ball on the same way,** and the result leans its way: the Galton board's one metering wheel made the first pin a switch (moving a chute 1.5 mm flipped the lean); a funnel and neck hid it, until the bounce was changed and the lean changed sides. William asked whether the setup was asymmetric: turning the wheel the other way swapped every lean. Two mirror-image wheels, geared, fixed it (the mean within about 0.3 of 6 from a bounce of 0.3 to 0.55). If a result depends on a parameter that should only change its spread, look for an asymmetry; test by mirroring (`GB_LOG=3` shows how the balls leave the neck).
- **An invisible change is a different run.** Rewriting the Galton board's OpenSCAD sizes file (three empty lists added, nothing drawn differently) changed its tour's numbers (5.88/2.12 to 5.73/2.18); so does running it with an empty parts cache. The parts don't collide, but something about building them changes the order the solver meets the balls, and the board is chaotic. Neither run is wrong. Compare a change against the same cache, and keep a file's text unchanged when its meaning is. (bpp's commit 02f18d43, "Objects and constraints in the order added, not a QSet's", came in the same night and is probably the cause: a QSet's order follows memory addresses, which any new part moves. Not yet checked: rebuild and see whether the Galton tour's numbers change, and whether a cold cache still makes a difference.)
- **Check a document's length after a scripted edit.** A replace anchored on the wrong text cut this file from 616 lines to 330 in the mirror-wheels patch (2026-11-13): most of section 5's later lessons, all of section 6 and most of section 7 went, and nobody noticed for a day. Restored 2026-11-14 from

the version before, the later edits merged back in (a three-way merge with the cut file as the base). `wc -l` and the list of headings before and after.

- **Fair is not independent** (the Galton board, `FINDINGS.md` section 6): each pin split the balls 50/50, yet the bins came out wider than the binomial. William guessed why: a ball leaving a pin carries a bias to the next. Recording each ball's path (`GB_PATHS`, `paths.py`) measured it: the same way again two times in three, the first pair 98%. A deader ball remembered more (damping would have been the wrong fix), and taller rows forgot but let balls jump pins; no setting tried beat the equilateral lattice. When a count is wrong but every single step looks fair, measure the correlations between steps.
- **A full container looks like a jam:** balls left at the slide's level were sitting on a heaped, full drawer, and knocking did nothing. Size a container for a heap, not for flat packing.
- **A smooth 60-degree wall is a ramp:** a ball at the edge of a pin lattice slid down it, squeezing past pins through gaps 0.5 mm too small (Bullet's soft contacts let it). Half pins standing out of the wall stop it. And a pin a little more than a ball's radius from a wall cradles a ball against it: leave it out and put a half pin on the wall.
- **Tall stacks of balls jitter** at 20 solver passes (5 to 6 cm/s at the bottom of 36 balls, the solver's, not a movement; 60 passes bring it to 1): call such balls still below 8 cm/s rather than pay for more passes.
- **OpenSCAD outlines** that only touch a circle that is cut too (an outline run along a circle 1 mm inside it) leave slivers, and the extrusion "is not closed" (the walls of a whole tile went missing, silently, in an STL that was still watertight). Run such an outline well inside the circle, and check each tile's thickness, not only that it is watertight. Check the outlines for self-intersection when a size changes (a wider funnel's mouth went outside the walls it should meet).
- `HALL_START` lands on an exhibit's own view, not its first stop (the lectern); Shift+Tab from there goes to the previous exhibit. To see a first stop, start at the exhibit before and Tab (seen 2026-11-12 with the Pascaline and the Galton board alike).

6. How to make an exhibit

1. Pick it from `MACHINES.md`; it goes in its place in the list's walking order, which moves the bays after it along one (`hall.lua` EXHIBITS; README table; the READMEs' bay numbers and the PDF subtitles in `make-pdfs.sh`).
2. Prototype the mechanism in a scratch script first and measure it.
3. `exhibits/<Name>/`: the script runs on its own and in the hall (find its folder with the `DIR` loop the others use; `hallPath` for OpenSCAD's full path), uses `exhibit-table.lua` (the machine's footprint), the start hold, resting, a tour that prints `RESULT ... PASS/FAIL`, sliders, keys, help text and hover text.
4. `parts/*.scad` as the single source of geometry (or, as the Pascaline, a Lua geometry file that writes the SCAD); `make-stls.sh` with each part turned to print; check the STLs and overlaps.
5. Sweep friction and speed, record it in `FINDINGS.md`; `README.md` with the parts table, what to buy, assembly; both PDFs; a `make-pdfs.sh` line.
6. In the hall: its EXHIBITS entry (`floorY = -100`), placard lettering (`make-labels.py`; a new key needs its line in `labels.lua` first), the lectern card (`make-lecterns.py`), `tools/check-all.sh` (add it there), screenshots.
7. **Make it show its working in the 3D view from the start**, not only in the console and tooltips: William watches in the GUI. Lehmer's sieve first showed nothing of N, x or the answer, and its probe's depth was hidden; it got a board of number drums (the exhibit's, said so) and a printed depth scale. When a machine runs on by itself, pause between steps so each reading can be seen (the sieve's

"through" drum was cleared in the frame it was set). Say what the machine computes and what the script does for the operator (the sieve only says "maybe"; the square test is Lua's).

7. Where things stand and what's next

- **Done (2026-10-21): Lehmer's number sieve** (then east 1, now west 4): four Geneva stars of 9, 8, 7 and 5 places on one axle, driven from one shaft, detents to centre them, pegged holes, a cam-lifted probe; it factors 1241 (and 1679, 1157, 2813: `N`) by Fermat's way. **2026-10-25:** William found it hard to see what it computed: now a board beside it (the exhibit's) shows N, x, each wheel's hole under the probe (green open, red pegged), the depth, and at a stop x*x - N, the verdict and the factors; a printed depth scale on the probe's guide reads how far the probe went; `T` runs the tour for the number `N` chose. The wheels' colours and the see-through cam are still first guesses. 2026-10-26:* the depth now stays on the board between turns, and the camera is further back; William's window is narrower than the virtual screen (about 1450 px of 3D view, not 1658), so frame views with room to spare. He asked whether the cam is one print: it is (one watertight solid); only the simulation's contact shape is 72 convex wedges. Offered, not done: finer cam steps (5 degree flats now). It is the dearest exhibit to step (a 25 s tour took about three minutes here, headless): if it is slow on his computer, `LS_HZ=1200` halves that and still passes.
- **Done (2026-10-26): the Digi-Comp II, step 1, the counter** (then west 3, later west 4, now east 4): three printed rockers, tipped over by the balls alone, and a feeder wheel. **The same day, step 2a, the accumulator,** which replaces it. William asked for the rest of the real machine; its manual (Evil Mad Scientist's copy, cdn.evilmadscientist.com/KitInstrux/DCII-manual.pdf) gives the registers: accumulator 7 flip-flops, memory 4 bits, MQ 3, distributor 3 (a ring counter), mode flip-flops T1 to T6 and complement CF1, CF2; switches MULTIPLY, CLEAR, COUNT, COMPLEMENT, OVERFLOW (run/halt) and AUTO/MANUAL; the ball's path INPUT, MULTIPLY, CLEAR, COUNT, then down the accumulator. Built now: a **two-level** board (balls that are done drop through holes to the lower level and roll to the collector), the 7-bit accumulator (each rocker has a foot on the lower level for CLEAR: a wedge a clear ball pushes back to 0), hand-set **switches** (flaps) for CLEAR and COUNT, and START (the feeder wheel, turned by hand). Keys: Space START, C and N the switches, 1 to 7 set a bit by hand. Board 322 x 878 mm, 10 upper and 10 lower tiles; table 100 x 150.
- **Done (2026-10-26, while William slept): step 2b, add and multiply.** The memory (four hand-set switches beside A1 to A4: 1 sends the ball into that flip-flop's funnel, by a long left wall the carry doesn't cross; 0 drops it), the MQ (three flip-flops read the other way, so a ball takes one away; a borrow out of it drops through STOP and the hand stops), the distributor (a tree of three flip-flops: the MQ and M4, M1, M3, M2 by turns; the manual's is a ring) and MULTIPLY, all thirteen flip-flops the same part. CLEAR's "on" outlet is on its right now (the memory's lane needs A1's left). Tour: count, clear, 37 + 11 = 48, 13 x 3 = 39, every ball checked; the hand puts balls back from the collector. Keys: M, C, N; 1 to 7; H J K L the memory; 8 9 0 the MQ; D the distributor. Board 424 x 1216 mm, 14 + 14 tiles; table 110 x 180. Not yet seen by William.
- **2026-10-27:** William asked for an illustrated manual of every operation, now and to come, and which parts each needs (for building part of the machine): `exhibits/DigiComp2/MANUAL.md`. Its figures are drawn from the geometry. Keep it up to date as steps are added; its section 6 (complement, subtract, divide, AUTO) gives E.S.R.'s ways and this board's.
- **Done the same night: the narrated tour.** Before each ball a yellow dashed line on the board shows the way it will go, and the sign's THIS BALL says what it will do and what A was and will be (`route()`, `preview()` in `digicomp2.lua`; also on `Space`). `DC_STEP` starts the tour at a later step, for testing.
- **Done (2026-10-27): step 2c, the complement: subtract and divide.** William chose the longer board (more visible) over a lower-level inverting lane. Not E.S.R.'s T1 to T6, CF1, CF2: under each of

A1 to A6's done holes a **done lane** runs on into the next flip-flop's funnel, and six **shutters** (one bar, high over the walls: the COMPLEMENT switch, `X`) cover the holes, so a ball goes on through every flip-flop to A7 and turns each over. Complement = two balls, the hand moving COMPLEMENT between them (CF1 and CF2's job). Dividing: MQ 7, the hand watches OVERFLOW and stops at the end of that cycle; quotient and remainder from the cycles and A (MANUAL section 6; FINDINGS says why the end of the cycle). A's steps are 14.75 cm apart now (`G.ADZ`), all its funnels have both walls long. Tour adds the complement of 25, 12 - 4 and 13 / 5 (about 27 600 frames; check-all allows 29 000). Board 424 x 1573 mm, 18 + 18 tiles; table 110 x 220. `tests/complement.lua`. The camera stands farther back for the longer board.

- **Done (2026-10-28): the hall loads a bit at a time.** William found the hall slow to start. It is shown at once now, a sheet over each exhibit's bay, and the exhibits load one a frame (the one you're at first); the status bar says which. The time is OpenSCAD making parts on a cold cache (94 s for all five here; 1.8 s cached). The Digi-Comp's sign letters are in `parts/sign.scad` now, so changing its geometry no longer remakes them.
- **Done the same night: programs.** William saw that the procedures are the operator's part (the machine has no stored program: its switches are control lines, each ball a clock tick, the operator its control unit) and asked for them as scripts the exhibit reads and runs line by line, with a library. `programs/*.dc2` (listed in `programs/LIBRARY`): the keys, plus `SET`, `UNTIL STOP|OVERFLOW|CYCLE`, `CHECK`; the "program" slider and `G` run one, the listing on the program board behind (`B`), the line in the status bar; `DC_PROGRAM=all` runs the library (53 000 frames, 12 minutes here). MANUAL section 9. (The hall's and the exhibits' status lines were never shown: bpp's `statusText` is a property, not a `setStatusText` method; the hall passes it on now.)
- **Then: the program board.** A board on a stand behind the machine shows the program, its line lit, scrolling with it; `B` flies the camera to it and back (set the camera's up vector, then its place, then its look, as separate statements: Lua's multiple assignment has no order).
- **Then: the tour is the library, the meter is off.** William asked for the tour to run the programs (so what starts by itself is a program, line by line), and for the hall's cost meter to be off unless asked for (the "Hall: meter" slider at the views, or `HALL_METER=1`; the freeze log goes with it). The Digi-Comp's tour is now ~53 000 frames; check-all allows 56 000 (and no longer runs the library separately).
- **Then: AUTO (step 2d).** The escapement is on the board (the knob is gone: the weight, a torque, turns the wheel, the fly caps its speed, the anchor holds it; Space pushes the START lever). Every ball goes down a trip channel at the bottom right, under a trip lever (an arm trailing down the slope from an axle: a ball can only lift it; its underside stays high near the axle, so a ball meets it far out), to the store. The axle comes out through the rim; an arm hangs from it beside the board, and a thread from its foot runs up beside the board to an eye and across to the START lever, which it pulls as the hand does. AUTO (`U`): the thread hooked in; MANUAL: off. STOP's ball drops into a pocket and trips nothing. Programs have `AUTO`, `MANUAL` and `RUN` (each ball checked at the lever, the next in flight); `multiply-auto.dc2` does 13 x 5 in 21 balls from one START. The library (the tour) is ~81 000 frames; check-all allows 86 000. FINDINGS, step 2d, has the tries: a lever reaching up the slope jammed; a push rod up the board worked but was too heavy to push unless balanced, and William pointed out a thin rod 1.5 m long can pull but hardly push, hence the thread. In the simulation the thread is inextensible and pull-only (impulses each step, `trip.pull`). The weight and fly as parts are not done.
- **Then: OVERFLOW's halt (step 2e).** OVERFLOW is a shutter over the overflow hole (key `O`, program step `OVERFLOW HALT|RUN`): RUN, the carry out of A7 rolls over it to the collector; HALT, it drops into a pocket and trips nothing, so AUTO stops. `count-out-auto.dc2` and `divide-auto.dc2` run E.S.R.'s way. A halt mid-cycle can leave A = 0 when a division isn't exact (11 / 5): the manual says to finish the cycle by hand. **Checked whole (2026-10-31):** check-all's eight runs of the other exhibits and `check-lua52.sh` passed, but both Digi-Comp runs gave no RESULT: the tour had not ended. Run

with its log, every program was right and the tour ended at frame 98 292, past the 96 000 check-all allowed (it had been put at ~88 000). check-all now allows 110 000 (110 600 in the hall), still two hours a run; each took about an hour here, two at a time on two cores.

- **Done (2026-10-31): the weight and the fly as parts (step 2f).** The wheel is glued on a 3 mm steel axle down through the plate and the board. Under the board (not in the lower level as planned: there it could not be wound, and the damper and ratchet had no room): a ratchet on the axle, the drum on the axle with two pawls in it, a coupler, and the **FRT-G2-200** damper screwed to a bracket. The cord runs up under the board to a pulley just out past the top edge and down to a printed cup of **eight** balls (74 g with the cup: torque 2.9 mN m, the old 30 kg cm^2/s^2); 57 balls from wound up to the table. `G.DRIVE` in the geometry; `drive` in the script (the cup a body, the cord pull-only as the thread, the damper a drag c w, `DC_DAMP` scaling it, `DC_CUP` the balls). The hand winds the weight when fewer than 25 balls are left and nothing moves; `R` leaves it as it is. The library passed at the damper's rating (on its own and in the hall) and at 0.65 and 1.35 of it (its tolerance), the hand winding three times a tour. Found: the anchor's second finger comes off its pin slowly against the damper (up to 1.8 s at its rating, 3.6 s at 1.35, where the hand's 3 s wait for the anchor was too short and readings came a ball early: it waits 8 s now); at 0.65 the wheel runs past the damper's 50 rpm. The damper's mount is drawn from our reading of an unlabelled datasheet drawing: measure its shaft before printing the coupler. FINDINGS, step 2f. **Not seen by William yet.**
- A possible tighter rule for dividing in AUTO (A = 0 at a halt is exact only if M's bits still to come in that cycle are all 0) was offered to William, not yet taken.
- **Done (2026-10-31, evening): each program with only its parts.** William asked for it on the full board, the parts "really absent" (out of the simulation, not only hidden). Every wall and peg has a `part` in the geometry (A, CLEAR, COUNT, MULTIPLY with D, MQ and STOP, MEMORY, COMPLEMENT, OVERFLOW, AUTO; BOARD and FEEDER always), the script tags the moving parts, and each program has a `PARTS` line; a step needing a part it doesn't name is an error when it is read. A program's start takes the rest off (`parts` in the script: fixed things moved far below the floor, moving ones frozen, untouchable and moved there too); they stay off after it, keys for them refuse, `R` by hand puts all back. `manual/fig-parts.png` is drawn from the same tags. Found by it: the clear lane's walls also steer A's done balls into the trip channel (without them AUTO stopped), and the done lanes' first stretch is the done holes' backstop (without it a fast carry skipped A2's hole and rolled off the plate): both are A's now. Watching OVERFLOW needs only A's hole and pocket; its shutter is for halting AUTO. **Not seen by William yet.** He pointed to a replica: www.instructables.com/Digi-Comp-II-Replica/ (only its summary could be fetched here).
- **Done (2026-11-01): the hand, shown.** William asked to see it: a drawn pointing hand (`parts/hand.scad`, its own file so the machine's parts aren't remade) that goes to everything the operator does (switches, flip-flops, knobs, START, the thread's hook, carrying balls from the store and pockets to the top, winding under the board) and does it on arrival; `hand` in the script, a queue of jobs, the machine busy while it has any. The tour takes 23% longer (it ends near frame 121 000); check-all allows 135 000. FINDINGS, "The hand, shown". Seen by William (2026-11-03): he asked to lose the forearm and keep the palm off the fingertip (done), and for `Q`, the hand off (done: at once and unseen, as before; `DC_HAND=0`).
- **Slow? Check POV-Ray export first (2026-11-03).** The machine crawled in William's hall, 2 s a frame of "physics" on the hall's meter: bpp's POV-Ray export was on (toolbar button or `P`; a cyan dot top right). It runs every frame before the steps, so the meter counts it as physics. A fix to bpp's export (format a mesh only when its `.inc` is missing) went to Jakob separately. FINDINGS, "How fast it runs".
- **The Shortcuts pane (2026-11-04):** William found he had to scroll to the keys. Every exhibit's pane now has its KEYS first, under one line of the hall's (Tab, Shift+Tab; where its sliders are moved to the

end); the Digi-Comp's no longer lists its library or the program (one line names it; the listing is on the program board, which the "program" slider now shows too). Resting the mouse on a table says what its placard says (`EXHIBIT_TABLE.parts`): the placards are low down and seldom in view. A new bpp pane was considered and set aside (screen room; bpp's C++).

- **Lecterns (2026-11-05):** a museum label on a brass post just right of an exhibit's table and beyond it, its card tilted up: the name, a line, a few sentences (`make-lecterns.py` writes the card as a PNG; the hall puts it on a thin box as a texture and builds the lectern when the card is there). `I` looks at it close up and back; the mouse on it says what the placard says. The Differential has one, which William chose (A) over a smaller card on the table beside the machine (B: in the exhibit's own view, but the view is tight round the machine, so it was cut off and crowded it). Where it stands went round three times: in front of the table (out of the exhibit's view); beyond the table's back right corner (turned 35 degrees to the aisle: away from the viewer; then turned to the camera, but far back, and it would be farther still at the Digi-Comp). William's idea settled it, **two stops at an exhibit with a lectern**: the first Tab walks up to it, a view the hall works out from the table and the lectern (eyes 162 cm up, square to the table, back far enough for both: `g.approach`), the lectern at the table's right front corner (`LECTERN`: 12 cm past its right edge, 14 cm before its front) turned to face that view; the next Tab steps in, to the exhibit's own view; then on. Shift+Tab the same, back (`steppedIn` in `hall.lua`). Stepping in or back flies straight (at first it went by the aisle, the way between bays, and William saw it back out and come in again). The exhibit is awake from the first stop (its tour, keys, pane). Only exhibits with a card have the two stops. Two camera faults found on the way, both old: the hall stood the camera upright before every flight, which turned it level, so a flight began by looking up (William saw stepping in pan up and back down); and an exhibit loading in the background still reached the real camera (standing it upright as it loaded), which now and then left you looking level where you were. Flights now start from the way the camera looks, and a loading exhibit touches the camera only to set its own view (the camera's pitch logged every frame: steady). All five exhibits then on show have a card (2026-11-06; wording William read first; the Domino Half Adder's names Matt Parker's 10,000-domino computer, Manchester Science Festival, October 2012, which adds two four-bit numbers: Think Maths' own page). `make-lecterns.py` shrinks a title or line too wide for the card and says if the text runs off its foot. Each exhibit's first stop seen in the window: the table, the lectern and the placard in view; the Domino Half Adder's big table puts its card farther off (zoom, or `I`). An exhibit added later gets the two stops by getting a card. **2026-11-11:** walked to with Tab, the sieve had no lectern: an exhibit loading while you're at another has that one's world in use, and its lectern went there (the Differential's showed only because it loads first; starting at the sieve hid it). The lecterns are the hall's now, in world 0 like the placards.
- **Checked against bpp's new Params and ScriptKeys (2026-11-11):** `tools/check-all.sh` all passed on Jakob's rewrite, and in the window Tab, the panes and the sliders work.
- **Done (2026-11-12, revised 2026-11-13): the Galton board** (now west 3), in a new row of bays: the hall has ten now, the planimeter's covered bay (west 2) before it (by rank, 14 and 13), and everything after it one row on, on the same side (Pascaline west 3, Domino Half Adder east 3, Digi-Comp II west 4, tide predictor east 4, Differential Analyser west 5, Difference Engine east 5; the READMEs' bay numbers and the PDF subtitles with them). 150 steel balls (6 mm), 4 mm pins 12 mm apart, twelve rows from the first pin, thirteen bins with the binomial's heights for 150 engraved in gold; a hopper, **two mirror-image metering wheels** geared together behind the board and turned by one crank (the hand, a motor; the gears, a lock holding both to the crank's turn), a funnel under both and a straight neck onto the first pin; a slide under the bins, a drawer, the hand tipping the drawer back into the hopper (the pouring moves the balls, as the Digi-Comp's hand does), and knocking the board if balls stay wedged (idealised; not needed since the drawer was made deeper). A sign (the exhibit's) shows the run, the balls in the bins, the mean bin and the spread against 6.00 and

1.73. The tour: a run, empty and pour, a run (about 4 min headless; check-all allows 6000 frames, 6600 in the hall). The first patch (2026-11-12) had one wheel, and the board leant whichever way the bounce tipped it: William spotted the asymmetry; the second patch replaces the wheel with the pair. Over six runs at the usual settings the mean bin was 5.97 and the spread 1.94 (fair coins: 6.00, 1.73); fair from a bounce of 0.3 to 0.55 and at friction 0.15 and 0.45. At a bounce of 0.7 the spread passes the tour's 2.6. The bounce (0.4, ball on board) is a guess (no figure for steel on printed PLA was found); FINDINGS says how to measure it. Keep the crank to 0.75 turns a second: faster, two balls meet over the neck and jam. Why wider than fair coins (2026-11-13, a third patch): each pin is fair but remembers the last (the same way again two times in three; FINDINGS section 6, with the bounce, friction and row-spacing sweeps); the board was left as it is, the instrument kept (`GB_PATHS`, `paths.py`, and `GB_GEO="V=..."` for the rows). Left to try: feeding the first pin a little off-centre, since the first pair goes together 98% of the time. Splitters (2026-11-13, a fourth patch): a neck above every pin, tested on its own (`splitter-test.lua`, `SPLITTERS.md`), for a second board that would show independent coins exactly and, with other walls, gambler's ruin and first passage. At a 30 mm pitch with 10 mm necks the decisions cannot be told from fair coins; but the bounce moves it (0.3: 69% the same way). A fifth patch tried to take the bounce out (mixing chambers, leaning necks): neither did, and the memory turned out to hang on how a ball meets the neck's mouth (a seam there moved it by 0.13), so no simulation can promise a printed splitter's memory. The plan (a sixth patch): the whole printed tooth is what swaps, in variants of one height so the rows stay put; four sets (necks 10, 12, 14, 20 mm) cover bounces 0.3 to 0.6 within 0.07, tuned on the built board by the pile's spread. A swallowing wall in the test bed (walls.py) gives first passage and the reflection principle as the formulas do with tuned teeth (chi-square 11.6 on 11 df, 3,000 balls), and fails plainly mistuned; two walls give gambler's ruin (three starts, all passing). A gambler's-ruin board wants its walls 4 places apart (2% survive 12 rows; a quarter, a half, three quarters go right). The exhibit (an eighth patch): splitter-board.lua, run by galton.lua through a small BOARD interface (its geometry, its OpenSCAD files, its sign and verdict); every wall mode passes its tour by chi-square. Still to come: its bay in the hall, its placard and lectern card, check-all, its printed parts. Next was: the splitter board in bpp, with swappable wall panels (plain; swallowing, for first passage and the reflection principle; both, for gambler's ruin). Both boards are to be kept. Printed in three tiles under 200 mm, two gears, a stand, an acrylic front sheet; every STL checked watertight, and no two parts overlap at rest. `tools/check-all.sh` (twelve runs, the Galton board's two included) all passed on the revised board. **Not seen by William in the GUI yet.**

- **Done (2026-11-14): the hall re-ordered, the splitter board in it.** William found the old order (by interest alone) off: the Domino Half Adder, which does one thing once, stood sixth; Lehmer's sieve second. MACHINES.md now orders all 48 by what each asks of a visitor (the idea it computes, the mechanism, how much it shows), in five tiers, and the hall walks it: dominoes first, the Galton and splitter boards facing each other in the third pair. The splitter board has its placard, lectern card and check-all lines (fourteen runs).
- **The splitter board's other distributions (not yet done).** With fair splitters the board has shown the binomial, first passage, the reflection principle and gambler's ruin (SPLITTERS.md). Still to do, in the order suggested (each in `splitter-test.lua` first, against the exact counts, then as a wall mode or a board of its own): 1. **Reflecting walls: the pile goes flat** (moderate, about a session). A wall that sends a ball back: it must zigzag, half teeth on it, as the pins sit at alternate places in alternate rows (the Galton board's half pins are the same idea). The pile flattens only after about the width squared in rows: walls 4 places apart want 16 to 20 rows, the board has 12 (37 mm a row: 0.6 to 0.75 m of rows), or narrower walls. 2. **Galton's two-stage board: conditional distributions** (moderate, about a session). His own 1877 design: one tall board, a row of compartments halfway down with a slide under them. Open all and the bell is the whole one; open one and a narrower bell stands under it (the root of regression). The parts exist; new are the compartments and their gate. 3. **First return:**

the Catalan numbers (moderate to hard, one or two sessions). Balls caught at the starting column, mid-board, must leave without crossing live splitters: through a hole into a second layer behind (the Digi-Comp II's two-level board is the precedent), ramps to the tubes there. For 150 balls about 75, 19, 9, 6, 4, 3. 4. **Splitters in two directions: the Rayleigh ring** (hard; an exhibit of its own). A 3D forest of splitters, ring bins; simulable, hard to print and build. The biased-pin distributions (Poisson, geometric, two humps, the Ehrenfest urn) were set aside at William's word (fair splitters only, so that the mathematics shows, not a construct). The arcsine law has no mechanism yet. **The splitter board is not seen by William in the GUI yet.**

- **Next (William, 2026-11-14): build the machines from the start of the list on.** Open: whether the first four (the abacus, the slide rule, Napier's bones, the combination lock: little or nothing to simulate) get bays, as simple hand-worked pieces at the start of the walk, or the building starts at the bead and spaghetti sorts (5), the first with real physics. Ask William first. Each new machine goes in its place in the walking order, the bays after it moving along. The covered bays (the planimeter, the tide predictor, the Differential Analyser, the Difference Engine) wait for their turn in the list. Still offered, not asked for: printing only a Digi-Comp program's parts (the tiles carry every wall; `make-stls.sh` could take a `PARTS` list, with tiles cut anew). Skipped, at William's word: fixing bpp's own `demo/koppi/gears.lua` (its `assign()`, which newer OpenSCAD dropped).
- **Settled (2026-10-21):** William has tried the Pascaline (the mouse and the keys) and it works well; the stylus collision he thought he saw wasn't one. Euler's Gallery keeps its own tables as they are; the exhibit standard is the hall's. The Rec Room's camera flights look good, the top-down table view included.
- **Open, for later:** POV-Ray finds textures only with `+L` the bpp folder (their paths are relative to it): Jakob has it. A printed detent (`detent.stl`, 1.2 mm leaf) would test the stiffness worked out from PLA; William will print one some other time.

8. Opening message for the next chat

> Clone github.com/bullet-physics-playground/bpp, check out the branch test-harness, and read demo/WyomingWill/BabbagesHall/HANDOFF.md, then MACHINES.md and README.md there. Build bpp as described, run tools/check-all.sh, and continue with HANDOFF.md section 7: [the next step: the splitter board's reflecting walls, or the first machine on MACHINES.md's list]. Deliver each step as a patch on test-harness, with its documents and PDFs, and the git commands.